

Homework 5

By Andrew McLaughlin

Problem 1

(c)

Estimate $C(S_0)$ using Monte Carlo simulation of S with 100 timesteps on $[0, T_1]$. Choose the number of paths large enough that the standard error (the sample standard deviation divided by the square root of the number of paths) is less than 0.05. Report the standard error. Do not use any variance reduction technique.

```
In [1]: import numpy as np
```

```
In [2]: # Exponential Ornstein-Uhlenbeck process
```

```
class XOU:

    def __init__(self, kappa, alpha, sigma, S0, r):

        self.kappa = kappa
        self.alpha = alpha
        self.sigma = sigma
        self.S0 = S0
        self.r = r
```

```
In [3]: hw5dynamics=XOU(kappa = 0.472, alpha = 4.4, sigma = 0.368, S0 = 106.9, r = 0.05)
```

```
In [4]: class CallOnForwardPrice:
```

```
    def __init__(self, K1, T1, T2):

        self.K1 = K1
        self.T1 = T1
```

```
self.T2 = T2
```

```
In [5]: hw5contract=CallOnForwardPrice(K1 = 103.2, T1 = 0.5, T2 = 0.75)
```

```
In [6]: class MCEngine:
```

```
    def __init__(self, N, M, epsilon, seed):

        self.N = N    # Number of timesteps on each path
        self.M = M    # Number of paths
        self.epsilon = epsilon # For the dC/dS calculation
        self.rng = np.random.default_rng(seed=seed) # Seeding the random number generator with a specified number hel

    def price_call_XOU(self, contract, dynamics):

        # You complete the coding of this function
        # self.rng.normal() generates pseudo-random normals

        dt = contract.T1 / self.N
        kappa, alpha, sigma, r = dynamics.kappa, dynamics.alpha, dynamics.sigma, dynamics.r
        K1, T1, T2 = contract.K1, contract.T1, contract.T2

        # Draw all normals at once; reuse for bumped path
        Z = self.rng.normal(size=(self.M, self.N))

        # Euler-Maruyama on X = Log(S) for baseline and bumped initial condition
        X      = np.full(self.M, np.log(dynamics.S0))
        X_bump = np.full(self.M, np.log(dynamics.S0 + self.epsilon))

        for i in range(self.N):
            shock      = kappa * dt * (alpha - X)      + sigma * np.sqrt(dt) * Z[:, i]
            X          += shock
            shock_bump = kappa * dt * (alpha - X_bump) + sigma * np.sqrt(dt) * Z[:, i]
            X_bump     += shock_bump

        # Closed-form forward price F_{T1} from XOU model
        tau = T2 - T1
        A = np.exp(-kappa * tau)
        B = alpha * (1 - A)
        C_const = (sigma**2 / (4 * kappa)) * (1 - np.exp(-2 * kappa * tau))
```

```

F      = np.exp(A * X      + B + C_const)
F_bump = np.exp(A * X_bump + B + C_const)

# Discounted payoffs
discount = np.exp(-r * T1)
payoffs   = discount * np.maximum(F      - K1, 0)
payoffs_bump = discount * np.maximum(F_bump - K1, 0)

call_price      = np.mean(payoffs)
standard_error  = np.std(payoffs, ddof=1) / np.sqrt(self.M)
call_delta      = (np.mean(payoffs_bump) - call_price) / self.epsilon

return(call_price, standard_error, call_delta)

```

```
In [ ]: hw5MC = MCEngine(N=100, M=100000, epsilon=0.01, seed=0)
```

```
In [14]: (call_price, standard_error, call_delta) = hw5MC.price_call_XOU(hw5contract, hw5dynamics)
```

```
In [ ]: print(call_price, standard_error, call_delta) # M = 1000
```

```
8.156718203830092 0.45722988478915694 0.3388727731168828
```

```
In [ ]: print(call_price, standard_error, call_delta) # M = 10000
```

```
7.934679927531252 0.13600287553961005 0.3456353694952341
```

```
In [15]: print(call_price, standard_error, call_delta) # M = 100000
```

```
7.732260523732917 0.042104579842347944 0.33959098927081044
```

After testing M at the M = 1000, 10000, and 100000 level, the M= 100000 level provided a standard error under .05.

(d)

Estimate $\partial C / \partial S$ by using Monte Carlo simulation to calculate

$$\frac{C(S_0 + 0.01) - C(S_0)}{0.01}$$

For the $C(S_0 + 0.01)$ calculation, reuse the same normal random variables which you generated for the $C(S_0)$ calculation. (Do not re-generate random variables to compute $C(S_0 + 0.01)$.)

```
In [16]: print(call_delta)
```

```
0.33959098927081044
```